Homework 6

Important Note: Do not distribute or share these problems. They are strictly for use by students registered in the Spring 2025 edition of CIS 3200 at University of Pennsylvania. Any violations will be referred to the Office of Student Conduct.

Note: The homework is due on **Monday, April 28 by 3:30 pm via Gradescope.** If you are using one of the two allowed extensions, then it is due on Wednesday, April 30 by 3:30 pm.

Please write concise and clear solutions; you will get only a partial credit for correct solutions that are either unnecessarily long or not clear. Whenever you present an algorithm, first give a precise description of your algorithm in English and only then present it in pseudocode if you think it is helpful. For most problems, pseudocode will be unnecessary. You should always argue the correctness of your algorithm and give an analysis of its complexity. Do remember to *typeset your homework*.

You are allowed to discuss ideas for solving homework problems in groups of up to 3 people but *you must write your solutions independently*. Also, you must write on your homework the names of the students with whom you discussed.

Finally, you are not allowed to use *any* material outside of the class notes and the textbook. Any violation of this policy may seriously affect your grade in the class.

Problem 1. Consider an instance of the Satisfiability Problem, specified by clauses $C_1, \ldots, C_m$ over a set of Boolean variables $x_1, \ldots, x_n$. We say that the instance is *monotone* if each clause is a disjunction of positive literals. Monotone instances of Satisfiability are easy to solve: they can always be satisfied by setting each variable to `TRUE`. For example, suppose we have the three clauses

$$(x_1 \vee x_2), (x_1 \vee x_3), (x_2 \vee x_3)$$

This is a monotone instance, and indeed the assignment that sets all three variables to `TRUE` satisfies all the clauses. However, observe that this is not the only satisfying assignment; we could set x_1 and x_2 to `TRUE`, and x_3 to `FALSE`. For any monotone instance, it is then natural to ask what is the smallest number of variables that are needed to be set `TRUE` in order to satisfy it.

Given a monotone instance of Satisfiability, together with an integer k, the problem of *Monotone Satisfiability with Fewest True Variables* asks: Is there a satisfying assignment for

the given instance in which at most k variables are set to TRUE? Prove that this problem is NP-hard by a reduction from the Vertex Cover problem. (**20 points**)

Problem 2. In the SPANNING TREE PACKING problem, you are given an undirected graph $G(V, E)$ and two integers k and ℓ, and the goal is to decide if there exists a subset $F \subseteq E$ of at most k edges in G such that there are at least ℓ distinct spanning trees of G that *only contain edges in* F. Note that two spanning trees of G are considered distinct if they differ by at least one edge. Prove that SPANNING TREE PACKING is NP-hard by doing a reduction from the Hamiltonian Cycle problem. (**20 points**)

Problem 3. You are given a puzzle consisting of an $m \times n$ grid of squares, where each square can either be empty, occupied by a red stone, or occupied by a blue stone. The goal of the puzzle is to remove some of the stones so that the remaining stones satisfy two conditions: (1) every row contains at least one stone, and (2) no column contains stones of both colors.

It is easy to see that for some initial configurations of stones, reaching this goal is impossible. We define the Puzzle problem as follows. Given an initial configuration of red and blue stones on an $m \times n$ grid of squares, determine whether or not the puzzle instance has a feasible solution. Prove that the Puzzle problem is NP-complete by using a reduction from the 3-SAT problem. (**20 points**)

Problem 4. You are given a set of n loaves of bread and a positive integer array $W[1..n]$ such that $W[i]$ is the weight of the i^{th} loaf of bread in ounces. Your goal is to pack bread into boxes so as to *maximize* the number of boxes that contain *at least* M ounces of bread. Design a polynomial-time 2-approximation algorithm for this task.
Hint: First consider the case where every loaf of bread has weight at most M.
(**20 points**)

Problem 5. The goal of this problem is to design a polynomial-time approximation algorithm for yet another *load balancing* problem. You are given a set of n tasks $t_1, t_2, ..., t_n$ to be scheduled on a set of m machines. For each task t_i, there is a non-negative processing time p_i — the amount of time it takes to complete the task on any machine. Your goal is to schedule the tasks on the machines such that the time T by which all tasks are completed is minimized. A machine can perform only one task at a time, and a task once started on some machine, must be completed on the same machine without interruption. Each task can be assigned to any machine. This load balancing problem is known to be NP-hard. So we will design an approximation algorithm.

Consider the following greedy algorithm. Set a counter $i = 1$. Assign the task t_i to a machine whose total assigned processing load is minimum. Increment i, and continue until all tasks have been scheduled. Let T^* denote the time by which all tasks are completed in an optimal schedule, and let T denote the time by which all the tasks are completed in the greedy schedule above. Prove the following assertions:

(a) $T^* \geq \max_{1 \leq i \leq n} p_i$. (**5 points**)

(b) $T^* \geq (\sum_{i=1}^{n} p_i)/m$. (**5 points**)

(c) Using (a) & (b), show $T \leq 2T^*$, that is, the greedy algorithm gives a 2-approximation. (**10 pts**)